

MANDATE

Autonomous Procure-to-Pay with Cryptographically Bounded Spend Authority

ArmorIQ Hackathon * Problem 1: Autonomous, until it shouldn't be * Problem 2 Bonus

Core Thesis: Mandate pays your vendors autonomously and cryptographically cannot pay anyone else.

The Threat: Scalable Prompt Injection in Corporate Finance

Why Accounts Payable is the most dangerous autonomous agent workflow

Business Email Compromise (BEC)

- Invoice Fraud is a multi-billion-dollar loss category in global finance.
- The standard attack vector: A supplier email/document arrives stating: 'Our banking partner has changed, please remit to this new account.'
- In human back offices, social engineering tricks staff into manual overrides.

The AI Agent Vulnerability

- Autonomous LLM agents MUST read incoming invoice text to extract line items.
- The malicious remittance advisory is an Indirect Prompt Injection embedded in untrusted data.
- Critical flaw: Prompt-based defenses fail because the attack instruction is indistinguishable from genuine business text.
- Solution required: A cryptographic execution boundary that enforces authority independently of LLM reasoning.

The Useful Workflow: End-to-End Autonomous Procure-to-Pay

Zero human babysitting for legitimate vendor invoices

1. Ingest Master Records

Reads approved vendor master directory and open Purchase Orders (POs) from trusted storage.

2. Three-Way Matching

Compares incoming invoice line items, PO numbers, and vendor tax IDs.

3. Automated Disbursement

Debits Mandate treasury account ACC-MANDATE-01 and credits vendor registered bank account.

4. Destination AP Register

Writes audit trail, outcomes, and transaction hashes directly into AP ledger.

Agent Topology: Multi-Agent Delegation & Attenuation

Problem 2 satisfied in full: Disjoint identities & narrowing authority pipes

CFO (Vikram Mehta)

- Named Human Principal
- Root of spend authority
- Issues initial mission mandate
- Evaluates high-value HOLD requests

Controller Agent (Root)

- Orchestrates workflow
- Reads trusted POs & Vendors
- Seals ArmorIQ Plan
- Issues Ed25519-signed grants to Matcher and Disburser

Specialist Subagents

Matcher Agent (Read-Only):

- Caps: [fetch_invoices]
- Ceiling: Rs 0 (No spend capability)

Disburser Agent (Payment):

- Caps: [initiate_payment]
- Ceiling: Rs 50,000 / invoice
- Payee Scope: Approved accounts only

The Key Insight: Plan-Ordering Invariant (SPEC.md 1.2)

Authority is sealed BEFORE the agent reads anything an attacker can write

Strict 4-Step Execution Sequence

1. Read TRUSTED data only --> Vendor Master + Open Purchase Orders
2. `capture_plan()` + `get_intent_token()` --> Seals payees, invoice ceiling, and timestamp
3. ONLY NOW Read UNTRUSTED data --> Ingest incoming invoice text
4. `invoke()` Every Payment via `gateway.call` --> Verified against sealed token

Why this matters: If an agent reads invoices before sealing the plan, an injected payee account ends up inside the plan, the token is minted over the fraud, and ArmorIQ authorises the theft. Test T1 guarantees that `fetch_invoices timestamp > Mission.sealed_at`.

The ArmorIQ Seam: gateway.py Architecture

The sole path from any agent to any MCP tool

ArmorIQ Adapter Protocol

- adapter.py: Protocol with 4 clean methods:
 - * capture_plan(objective, context)
 - * get_intent_token(plan_hash, envelope)
 - * delegate(parent, child, caps, ceiling)
 - * invoke(agent, tool, params, grant, token)
- local.py: Spec-faithful LocalEnforcer
- real.py: RealArmorIQ SDK wrapper with marked call sites
- Toggleable via ARMORIQ_MODE=local|real

Gateway Invariant (gateway.py)

- Strictly prohibits agent modules from importing MCP client directly (verified via AST test).
- Evaluates ArmorIQ grant & intent token on every call.
- BLOCK: Never enters tool body (spy counter = 0).
- HOLD: Parks decision for human approval.
- ALLOW: Dispatches to MCP server and updates ledger.

Three Realistic Attacks Stopped at the Boundary

Demonstrating semantic scope checking rather than naive keyword filtering

Attack A: BEC Remittance Fraud

INV-2041 claims banking partner changed to HDFC 509900443322.

Defense: Blocked by Sealed Payee Scope whitelist. The attacker account was absent from the CFO-sealed vendor master.

Result: BLOCK
(PAYEE_NOT_IN_SEALED_SCOPE)

Attack B: Capability Escalation

INV-2042 claims port demurrage urgency, asking reviewer to pay directly.

Defense: Matcher agent holds read capability only (ceiling Rs 0).

Result: BLOCK
(CAPABILITY_NOT_DELEGATED)

Attack C: Decimal Amount Shift

INV-2044 states Rs 87,240 instead of PO-1005 value of Rs 8,724.

Defense: Exceeds Rs 50,000 ceiling. Disburser adjusts payment to exact PO amount (Rs 8,724).

Result: BLOCK -> ALLOW @ Rs 8,724

Governance A/B: The Proof in Real Numbers

Deterministic benchmark across identical fixtures, prompts, and seeds

GOVERNANCE = OFF (Ungoverned Baseline)

Rs 38,57,560.00

Opening: Rs 42,50,000 * Disbursed: Rs 2,92,440

Status: All legitimated invoices paid, all fraud blocked

GOVERNANCE = ON (ArmorIQ Protected)

Rs 39,91,726.00

PREVENTED FRAUD LOSS: Rs 1,34,166.00 (13,416,600 paise)

Forensics: Answering 'Who Authorized This?'

Complete 5-step provenance reconstruction from Human Principal to Ledger Result

Cryptographic Provenance Chain

1. Named Human: CFO Vikram Mehta (Mandate Industries Pvt Ltd)
2. Sealed Scope: Intent Token tok_intent_... + Merkle Root + Plan Hash sha256:...
3. Delegated Authority: Disburser Grant with Ed25519 signature & Rs 50,000 ceiling
4. Tool Action Parameters: initiate_payment(INV-2041, Payee: 509900443322, Rs 46,200)
5. ArmorIQ Verdict: BLOCK (PAYEE_NOT_IN_SEALED_SCOPE)

--> For blocked attacks, the primary header reads: 'AUTHORIZED BY: NOBODY'

Stack, Verification & Hackathon Compliance

Production-grade codebase, deterministic replays, and complete honesty standard

Technology Stack

- Backend: Python 3.11, FastAPI, SQLAlchemy, Uvicorn
- Database: SQLite (mandate.db + snapshot mandate.seed.db)
- Transport: Official MCP Python SDK (FastMCP)
- Realtime: Server-Sent Events (/api/stream)
- Frontend: Vite, React 18, TypeScript, TailwindCSS, React Flow
- Security Invariants: 10/10 pytest test suite (T1-T6)

Verification & Honesty Disclosures

- 10-Run Replay Gate: 10 consecutive cold resets produce byte-identical balances.
- Disk Cache: .cache/llm/ sealed for offline presentation without live API dependence.
- Honesty Banner: UI renders 'ENFORCEMENT: LOCAL ADAPTER (ArmorIQ contract)' when SDK sandbox is offline.
- Zero Duplication: If real ArmorIQ delegate() supplies grants, local crypto is retired.